

Design and Quality Review

1. Anomaly and Redundancy Analysis

The first redundancy in the schema is the duplication of the `requires_approval` flag. It is present in the `room` table (as the current room policy) and in the `booking` table (as a snapshot at the creation time of the booking).

This is a **reasonable denormalization**, but it still has the risk of an **update anomaly**: if the approval policy for a room changes, the `room.requires_approval` value is updated, but all historical `booking.requires_approval` snapshots remain unchanged. This is actually the intended behavior: historical bookings should not be changed retroactively. Future developers should not assume both columns are the same and try to synchronize them, corrupting the audit trail.

A secondary risk is in the `booking_series` table. It stores `room_id`, `organization_id`, and `created_by`. Each individual `booking` occurrence stores these same three foreign keys independently. If a booking later becomes associated with a different organization (for example, by correction), the series record and its occurrence records may become out of sync. There is no schema level constraint enforcing that values of `booking.organization_id` be the same as `booking_series.organization_id` for bookings belonging to that series, which is an **insertion/update anomaly risk** in the current design.

2. Functional Dependencies

FD 1: `booking_id` $\rightarrow$ `requires_approval` (on the `booking` table)

A booking's `requires_approval` value is determined entirely by its `booking_id`: it is a snapshot of the room's approval policy taken at the moment the booking was created.

This FD is significant because it sits in tension with the analogous FD `room_id` $\rightarrow$ `requires_approval` on the `room` table. Both columns carry the same semantic label but serve different purposes. The booking-level snapshot breaks the normal expectation that approval policy is a property of the room alone. This is the root of the update anomaly described above: if `room.requires_approval` is updated, there is no cascade to historical bookings, which is correct by design but must be enforced by application logic rather than the schema.

FD 2: (`room_id`, `equipment_id`) $\rightarrow$ `quantity` (on the `room_equipment` table)

The number of a particular type of equipment in a particular room is determined by the room/equipment type pair.

This FD shows that `room_equipment` is in **Boyce-Codd Normal Form (BCNF)**: the only determinant is the composite primary key (`room_id`, `equipment_id`), and `quantity` depends on the whole key, not a part. There is no anomaly here. The FD is noted because it shows why we kept `quantity` in the linking table rather than on `equipment_type`. If it were to move there, it would incorrectly imply that quantity is a property of the equipment type globally, rather than of a specific room's inventory.

FD 3: `series_id` → (`room_id`, `organization_id`, `created_by`, `day_of_week`, `start_time`, `end_time`) (on `booking_series`)

All descriptive attributes of a recurring booking series are determined by the series identifier alone.

This FD is clean and well-structured. However, it implies an implicit expected FD between `booking_series` and `booking`: for any booking where `series_id IS NOT NULL`, one would expect `booking.room_id = booking_series.room_id` and `booking.organization_id = booking_series.organization_id`. No CHECK constraint in the current schema enforces this expectation, which is the anomaly risk pointed out in Section 1. Theoretically, a booking occurrence could be inserted with a mismatched `room_id` relative to its parent series, and the schema would silently accept it.

3. Schema Revisions

The schema was reviewed against the three functional dependencies above and the anomaly risks identified. Two decisions were made:

No structural change to `requires_approval` duplication. The snapshot pattern is retained as-is. The risk is real but the design intent is correct: `room.requires_approval` reflects current policy; `booking.requires_approval` is an immutable historical record. The CHECK constraint on `booking` already enforces consistency between `requires_approval`, `approval_granted`, and `status` at the row level, which is the appropriate place for this logic. Adding a trigger to copy `room.requires_approval` into `booking.requires_approval` at INSERT time would be a reasonable application-layer safeguard, but is outside the scope of the schema definition itself.

A note is added regarding series-occurrence consistency. The schema does not currently enforce that `booking.room_id` matches `booking_series.room_id` when a `series_id` is present. In a production system, this would be addressed with either a trigger or an application-layer validation. For the scope of this project, the risk is documented as a known limitation rather than resolved structurally, since adding a cross-table CHECK

constraint enforcing this relationship is not supported natively in PostgreSQL without a trigger.

The `booking_series.organization_id` column is correctly nullable (to match `booking.organization_id`, which is also nullable: a person may book without an organization). This was verified to be consistent across both tables in the current schema.

4. Privacy Review

The `person` table stores only a name and a unique email address, the minimum required to identify who created a booking and to enable contact if a booking needs to be managed or cancelled. No phone numbers, student IDs, roles, or affiliations are stored, keeping personal data to a minimum. The `cancellation_reason` field on `booking` is the most significant privacy risk in the schema: it is a free-text field with no length constraint or content validation, and a user could enter personal or sensitive information (for example, a medical reason) that the system has no mechanism to detect or restrict. This field should be treated as potentially sensitive personal data in any access control policy, and operators should be advised to keep entries brief and non-identifying. Finally, because `person_id` is referenced by both `booking` and `booking_series`, deleting a person record is blocked by foreign key constraints. The system should define a clear data retention and deletion policy to handle right-to-erasure requests without orphaning booking history.

5. Rationale

Overview of the Design

The central idea of this design is to separate intent from occurrence. A student organization that books a room every Wednesday evening for a term has one planning intent but many individual time slots. The schema reflects this directly: `booking_series` stores the intent (the pattern, the room, the organization, the date range), while each `booking` row stores one concrete occurrence. A singular booking is simply a booking with no parent series (`series_id = NULL`). This separation means the system can answer both "does the organization hold Wednesday evenings this term?" and "was the 14 March session cancelled?" without conflating the two questions. Every other table in the schema supports this core structure: `building` and `room` describe the physical space being reserved; `organization` and `person` describe who is doing the reserving; `equipment_type` and `room_equipment` describe what is available in each space; and the `booking` table, with its status, approval, and cancellation fields, records what actually happened.

The Most Difficult Modeling Decision

The hardest decision was how to model the approval state. The case says only that some rooms require special access approval, and that the system should record whether approval

was required and whether it was granted. It says nothing about who approves, how many approval steps exist, or whether approvals can be partially granted.

We chose the simplest representation: a single nullable boolean `approval_granted` paired with a `requires_approval` flag. This captures all four meaningful states (no approval needed, pending, approved, rejected) without introducing an approver identity, a separate approval table, or a multi-step workflow that the case never asked for. The logic is encoded in a CHECK constraint that ties these two columns to the booking's `status`, making illegal combinations impossible at the database level: a booking cannot be `confirmed` while approval is still pending, and cannot be `rejected` unless approval was explicitly denied.

The tradeoff is that the schema cannot answer "who approved this booking?" or "how long did approval take?" If those questions become relevant later, the design would need a separate `approval` table with its own audit trail. For now, the scope was kept tight to what the case actually requires.

Choice of Primary Identifiers

Every table uses a surrogate primary key. The alternative was to use natural keys: a room's name or number within its building, an organization's name, a person's email address. Natural keys were rejected for three reasons.

First, they are mutable. An organization can be renamed, a person can change their email address, and a room can be renumbered when a building is renovated. A surrogate key never changes, so foreign key references across the schema remain stable regardless of what happens to the real-world data. Second, natural keys are often composite. A room is only uniquely identified by the combination of its building and its name or number, and using this as a primary key would propagate a two-column key into every table that references rooms, making joins more verbose and indexes larger. Third, surrogate keys keep the schema honest about what is actually unique: the `UNIQUE` constraint on `(building_id, name_or_number)` in the `room` table still enforces that no two rooms in the same building share an identifier, but this is a business rule separate from the row's identity. The same pattern applies to `organization.name` and `person.email`, both of which carry `UNIQUE` constraints independently of their surrogate keys.

Where Normalization Shaped the Design

The clearest example of normalization driving a structural decision is the extraction of `equipment_type` into its own table. Another approach would have been to store equipment as a comma-separated string on the `room` row, or to add boolean columns like `has_projector`, `has_whiteboard`. Both approaches violate First Normal Form: the first by storing multiple values in a single column, the second by encoding a variable-length list as a fixed set of columns that would require a schema change every time a new equipment type is introduced.

Extracting `equipment_type` into a standalone table and resolving the many-to-many relationship through `room_equipment` puts the design in Third Normal Form. The `quantity` column on `room_equipment` depends on the full composite key (`room_id`, `equipment_id`), not on either foreign key alone. This means it belongs in the linking table, not on `equipment_type` (where it would imply a global quantity independent of the room) and not on `room` (where it could not vary by equipment type).

An Assumption Made Because the Case Was Incomplete

The case states that a recurring booking should be treatable "both as a single planning intent and as a series of individual occurrences", and that one occurrence may be cancelled or moved without affecting the rest of the series. It does not say what "moved" means in practice, whether a moved occurrence simply gets a new time, or whether it generates a replacement booking row, or whether the series itself is modified.

The assumption made here is that moving an occurrence is handled by cancelling the original booking row and creating a new standalone booking for the rescheduled time, with no changes to the `booking_series` record. This keeps the series record as a pure statement of original intent and treats each occurrence as an independent row that can be cancelled, confirmed, or rejected on its own merits. The consequence is that the series record will always show the original pattern, not reflecting specific reschedules. This was judged acceptable because the case frames the series as a planning intent, not a live schedule. If the university later requires the series record to stay in sync with actual occurrence times, a more complex event-sourcing approach would be needed, but that goes well beyond what the case describes.